

CUTTLEFISH_NEZHA_RUNBOOK

Helix OTA — Cuttlefish real-A/B bring-up RUNBOOK on nezha (operator-driven privileged steps)

Field	Value
Revision	2
Last modified	2026-06-23T00:00:00Z
Status	launch-command VERIFIED — the slim image is built+committed, the assets are staged+integrity-verified, and a rootless build-matched fetch-test PROVED launch_cvd runs and assembles the cvd config; WAITING ON the operator's one privileged sudo launch, then the agent drives A/B. Integration-pending until the privileged run + captured slot-flip/rollback evidence.
Status summary	The exact, ordered, operator-vs-agent step split for the real Cuttlefish (cvd) Tier-2 OTA run on nezha. nezha has no passwordless sudo, so every privileged step is fenced as an OPERATOR PRIVILEGED STEP the operator runs by hand; the agent drives every rootless/unprivileged step. The runtime model (file:// asset-feed via the mounted /staging, host-package-extracted-at-runtime launch_cvd) is now PROVEN by a rootless build-matched fetch-test (§4.5). HONEST BOUNDARY: this runbook still does NOT claim a real-A/B PASS. The asset-feed + launch_cvd discovery + config assembly are proven; only the privileged boot (which needs /dev/kvm + bridge, impossible rootless) remains. A real-A/B PASS is earned only by running §2.3 privileged + capturing slot-flip + auto-rollback evidence (§11.4.107/§11.4.108/§11.4.69).
Authority	Helix OTA control-plane / device-integration team
Related	docs/design/CUTTLEFISH_ROOTFUL_EXCEPTION.md (the §11.4.161 documented exception this runbook executes under); docs/design/CUTTLEFISH_TIER2.md (the Tier-2 recipe + sources); docs/qa/20260623-cuttlefish-launch-verified/REPORT.md (the §4.5 pre-verify proof evidence); tests/emulator/tier2_cuttlefish_ab.sh (the A/B + auto-rollback validation the agent drives); containers/pkg/cuttlefish/ (cvd-lifecycle wrapper + Containerfile + entrypoint.sh)

0. What this runbook is (and is NOT)

This is the **decision-A deliverable**: the EXACT step-by-step for the real Cuttlefish A/B run on nezha. It is the operational complement to docs/design/CUTTLEFISH_ROOTFUL_EXCEPTION.md (which *authorises* the container privilege) and docs/design/CUTTLEFISH_TIER2.md (which holds the cited research).

- **What it is**: an ordered checklist where the **operator** runs the few privileged steps (nezha has no passwordless sudo) and the **agent** drives everything rootless/unprivileged around them.
- **What it is NOT**: a claim that Tier-2 ran. No PASS is asserted here. Real-A/B fidelity is earned only by executing §§4–6 below and capturing the slot-flip + auto-rollback evidence. Until then this path is launch-command-verified + operator-privileged-launch-pending — not a green.
- **What is now PROVEN (FACT, §4.5)**: a rootless, build-matched fetch-test ran the entrypoint.sh file:// asset-feed end-to-end — it fetched the device image (super.img + boot/init_boot/vbmeta extracted), fetched the host package (./bin/launch_cvd present), and launch_cvd **RAN**, assembling the cvd-1 config with Launcher Build ID: 15660610, before the EXPECTED rootless VIRTUAL_DEVICE_BOOT_FAILED run_cvd returned 10 (no /dev/kvm/ bridge under rootless). The asset-feed + launch_cvd discovery + config assembly are **proven**; only the privileged boot remains.

Every step is marked [OPERATOR] (privileged, run by hand) or [AGENT] (rootless, the agent drives). The **only** privileged surface is module-load + device-node creation + the --privileged container run (§5 scope-narrowing in CUTTLEFISH_ROOTFUL_EXCEPTION.md).

1. Prereqs (assets staged + image built)

1.1 Verified nezha device state (FACT — host probe 2026-06-22)

Node / module	State on nezha	Consequence
/dev/kvm	present, world-rw	KVM available — passthrough OK
/dev/vhost-vsock	present, world-rw	vsock VMM path OK
/dev/vhost-net	present, world-rw	guest-net VMM path OK
/dev/net/tun	present, world-rw	tap-device backing OK
/dev/vsock	ABSENT	the --device /dev/vsock line is conditional (§2.1)
vhost_vsock / vhost_net kernel modules	not in lsmod — consistent with built-in to the Linux 6.12 kernel	UNCONFIRMED: built-in vs unloaded — VERIFY at §2.1 before the first launch (do not guess)

nezha is a Linux x86_64 host with KVM (62 GB RAM). Host arch x86_64 → Cuttlefish target aosp_cf_x86_64_only_phone-userdebug (no credentials, public ci.android.com) [see CUTTLEFISH_TIER2.md Sources].

1.2 Staged assets (FACT — ~/cf-staging/ on nezha, integrity-verified)

Build **15660610** aosp_cf_x86_64_only_phone-userdebug (no credentials, public ci.android.com). Both assets are staged + integrity-verified on nezha. The original curl download truncated img.zip mid-stream; it was recovered via a resumable wget -c and re-verified.

Asset	Size (bytes)	Integrity	Role
~/cf-staging/cvd-host_package.tar.gz	898828370	gzip-valid	the cvd host package — extracts to ./bin/ launch_cvd, ./bin/ adb, ./bin/ stop_cvd, ./bin/ update_device.py
~/cf-staging/ img.zip	1163637538	unzip / unzip -l-valid (boot/ init_boot/ vendor_boot/ super.img)	the device-image zip for the matching build

Both come from the **SAME** build 15660610 (the host package + device image **MUST** be from one build or launch_cvd mismatches) — this is the build the §4.5 fetch-test matched and proved against.

1.3 Container image build — [AGENT], rootless — DONE (slim image, committed)

Building the Cuttlefish container image needs **no privilege** (ordinary rootless podman build) per CUTTLEFISH_ROOTFUL_EXCEPTION.md §5(1). This is **DONE**: the **slim** image was built rootless on nezha at **1.11 GB** (vs 27.6 GB for the single-stage from-source build), via the upstream runner-prod **prebuilt-.deb** path — cuttlefish-base / cuttlefish-user **1.54.1** pulled from <https://us-apt.pkg.dev/projects/android-cuttlefish-artifacts/android-cuttlefish/main> (NO Bazel / cargo / from-source compile). cvd version 1.54.1 executes inside the image. Committed to the containers submodule at **54aa9b2**; parent pointer **659c2326**.

```
# [AGENT] rootless – on nezha, from the containers submodule root
# (the proven path)
cd ~/helix_ota/containers # or wherever the checkout lives on
                             nezha
podman build \
    -f pkg/cuttlefish/Containerfile \
    -t helix-cuttlefish:slim .
podman run --rm helix-cuttlefish:slim cvd version # => 1.54.1
# (FACT)
```

Runtime model (FACT — §11.4.28 decoupled). The image ships the modern cvd launcher (1.54.1). It does **NOT** bake the guest images or launch_cvd — launch_cvd comes from the **host package extracted at RUNTIME by the entrypt.sh**, which reads CF_HOST_PKG_URL / CF_IMG_URL (here file:// URLs pointing at the bind-mounted /staging). This keeps the image project-agnostic and lets the agent re-stage assets without rebuilding. The §4.5 fetch-test proved exactly this asset-feed path.

1.4 Rootless→rootful image transfer — save | load (FACT)

The image was **built rootless** but the privileged launch (§2.3) runs **rootful** (the §11.4.161 documented exception — privileged containers need root on nezha). A rootless-built image is not visible to the rootful podman store, so the operator transfers it via save → load before the run:

```
# [AGENT, rootless] save the built image to a tar (1.03 GiB on disk)
podman save -o /tmp/cf-slim.tar helix-cuttlefish:slim
# [OPERATOR, rootful — needs sudo on nezha] load it into the rootful store
sudo podman load -i /tmp/cf-slim.tar
```

2. OPERATOR PRIVILEGED STEPS — run by hand on nezha

OPERATOR PRIVILEGED STEPS — nezha has NO passwordless sudo. The operator runs the commands in this section by hand (each needs sudo). The agent CANNOT run these and will WAIT (honest §11.4.21 block) until they are done. These are the **only** privileged steps; everything else is rootless. Authorised by docs/design/CUTTLEFISH_ROOTFUL_EXCEPTION.md (§11.4.161 documented exception); high-blast-radius → operator-gated for first use (§11.4.101/§11.4.21).

2.1 [OPERATOR] Verify / load vhost modules + create /dev/vsock if absent

First **verify** whether the vhost modules are built-in (so modprobe is a no-op) vs loadable — do not guess (§11.4.6):

```
# [OPERATOR] verify-before-load (FACT-gathering, no change if built-in)
ls -l /sys/module/vhost_vsock /sys/module/vhost_net 2>/dev/null
# present => built-in or loaded
```

```
modinfo vhost_vsock 2>/dev/null | head
-1                                # 'name:' line => loadable
module
lsmod | grep -E 'vhost_vsock|vhost_net' || echo "not listed
(built-in OR unloaded)"
```

- If /sys/module/vhost_vsock AND /sys/module/vhost_net **exist** → built-in or already loaded → **nothing to load**, skip the modprobe.
- If they do **not** exist and modinfo shows a module → load them:

```
# [OPERATOR] load only if NOT already present (verified above)
sudo modprobe vhost_vsock
sudo modprobe vhost_net
```

Then handle the **absent /dev/vsock** client node. UNCONFIRMED: whether Cuttlefish on this kernel needs /dev/vsock in addition to /dev/vhost-vsock — the conservative move is to create it so the --device /dev/vsock line works; if the node still does not appear after modprobe vhost_vsock (the vsock core usually auto-creates it), create it manually:

```
# [OPERATOR] create /dev/vsock ONLY if still absent after
vhost_vsock load.
# The vsock misc device is char major 10, minor 53 on mainline
Linux -
# CONFIRM with `cat /sys/class/misc/vsock/dev` (prints
"MAJOR:MINOR") BEFORE
# the mknod, never hardcode-guess the minor (§11.4.6/§11.4.111
resolve-by-name).
ls -l /dev/vsock 2>/dev/null && echo "already present - no mknod
needed" || {
DEV="$(cat /sys/class/misc/vsock/dev 2>/dev/null)" # e.g.
10:53
echo "vsock misc dev = ${DEV:-<unknown - investigate before
mknod>}"
# Only if DEV resolved: sudo mknod /dev/vsock c ${DEV%*:}* $
{DEV##*:} && sudo chmod 666 /dev/vsock
}
```

Honest boundary (§11.4.6): prefer letting the vsock module auto-create /dev/vsock (load vsock if /dev/vsock is genuinely needed and absent: sudo modprobe vsock). Manual mknod is the fallback only when the kernel does not auto-create it; the major:minor MUST be read from /sys/class/misc/vsock/dev, never assumed.

2.2 [OPERATOR] (one-time) host group membership

If not already done (the Cuttlefish host packages add these via udev rules on a Debian host, but nezha runs the container path, so grant the launching user the groups the privileged container's --group-add mirror):

```
# [OPERATOR] one-time, then re-login (or reboot). Reversible via
gpasswd -d.
sudo usermod -aG kvm,cvdnetwork,render "$USER"
```

UNCONFIRMED: whether cvdnetwork/render groups even exist on nezha (they are created by the cuttlefish-base deb on a native install — on the container path they may be absent). VERIFY with `getent group cvdnetwork render kvm`; if a group is missing, the `--group-add <name>` in §2.3 simply has no host group to map — drop the missing ones from the run line rather than fail (§11.4.6).

2.3 [OPERATOR] The privileged container run — THE VERIFIED COMMAND

This is **the** §11.4.161 documented exception. Run the exact block below by hand on nezha (each line needs `sudo` — nezha has no passwordless `sudo`). It loads the rootless-built slim image into the rootful store (§1.4), then runs the privileged container with the `entrypoint.sh` file:// asset-feed (§4.5-proven). The agent then watches `podman logs` and drives the A/B run (§4).

```
# [OPERATOR] THE verified privileged launch on nezha (sudo
    throughout).
# - sudo modprobe vhost_vsock      creates /dev/vsock (absent by
    default, §1.1/§2.1).
# - sudo podman load              brings the rootless-built
    slim image into the rootful store (§1.4).
# - --privileged --network host   the §11.4.161 documented
    exception.
# - -v ../cf-staging:/staging:ro  bind-mounts the staged
    build-15660610 assets read-only.
# - CF_HOST_PKG_URL / CF_IMG_URL  file:// URLs => entrypoint
    extracts launch_cvd + super.img at runtime (§4.5-proven).
sudo modprobe vhost_vsock
sudo podman load -i /tmp/cf-slim.tar
sudo podman run -d --name cuttlefish --privileged --network host \
    --device /dev/kvm --device /dev/vhost-vsock --device /dev/vhost-
    net \
    --device /dev/vsock --device /dev/net/tun \
    -v /home/milosvasic/cf-staging:/staging:ro \
    -e CF_HOST_PKG_URL=file:///staging/cvd-host_package.tar.gz \
    -e CF_IMG_URL=file:///staging/img.zip \
    helix-cuttlefish:slim
sudo podman logs -f cuttlefish
```

Why the entrypoint launch (not sleep infinity)? The §4.5 fetch-test proved the `entrypoint.sh` asset-feed path works end-to-end (fetch image → fetch host package → run `launch_cvd` → assemble cvd config). Under privilege the same entrypoint reaches the boot stage instead of the EXPECTED rootless `run_cvd` returned 10. The operator watches `sudo podman logs -f cuttlefish` and hands off to the agent once the cvd is up (adb reachable). The `--device` set is exactly the §1.1-verified node set; `/dev/vsock` is created by the `modprobe vhost_vsock` on the first line.

Confirm it is up (operator can eyeball, agent re-checks):

```
sudo podman ps --filter name=cuttlefish      # STATUS "Up"
```

When the cuttlefish container is **Up** and `podman logs` shows the cvd booting, hand back to the agent — the privileged surface is done.

3. Handoff marker

After §2.3, the operator signals the agent (e.g. drops a marker the agent polls, or simply tells the agent "cuttlefish is up"). With the entrypoint-launch path the cvd boots inside the container automatically; the agent then watches `sudo podman logs -f cuttlefish` for the cvd to come up and adb-connects to it (the cvd exposes adb on the host via `--network host`). If the agent needs to drive steps **inside** the container (`podman exec`) and the rootful container is root-owned, that single exec invocation is the one place the agent needs the operator to have granted it (or the operator runs the agent's §4 driver under `sudo` once). **UNCONFIRMED:** whether the agent can `podman exec` into the root-owned container without `sudo` on nezha — establish at handoff (try `podman exec cuttlefish true`; if it fails with a permission error, the operator runs the §4 driver under `sudo`).

4. AGENT STEPS — drive the run inside the container (rootless where possible)

[AGENT] Everything in this section the agent drives. With the entrypoint-launch path (§2.3) the cvd boots automatically; the agent watches `podman logs`, adb-connects, and runs the validation harness against the running cvd. Where a step must run **inside** the container, the agent uses `podman exec` into the already-privileged cuttlefish container (or, if exec needs root on nezha, the operator runs the agent's §4 driver once under `sudo`). No NEW privilege is requested — the container is already privileged from §2.3.

Note (entrypoint vs manual extract): because §2.3 runs the entrypoint with the `file:// asset-feed` (§4.5-proven), the `extract + launch_cvd` of §4.1/§4.2 happen **automatically inside the container at boot** — the agent normally just watches `podman logs` and proceeds to §4.3. The explicit §4.1/§4.2 commands below are the manual fallback (e.g. if the entrypoint launch is skipped in favour of a `sleep infinity` PID-1 for step-by-step driving).

4.1 [AGENT] Extract the staged assets inside the container (manual fallback)

```
# [AGENT] inside cuttlefish – extract host package + device image
from /staging.
podman exec cuttlefish bash -lc '
    set -euo pipefail
    cd "${HOME:--/cuttlefish}"
    tar -xzf /staging/cvd-host_package.tar.gz
    ( cd "$PWD" && unzip -o /staging/img.zip )
    test -x ./bin/launch_cvd || { echo "ERROR: launch_cvd missing
        after extract" >&2; exit 1; }
    echo "extracted: $(./bin/launch_cvd --help >/dev/null 2>&1 &&
        echo launch_cvd-OK)"
    ,
```

UNCONFIRMED: the exact device-image filenames inside `img.zip` (`super.img` vs `system.img+vbmeta.img` etc.) and whether the build is **Virtual-A/B** or **legacy A/B** — read `getprop ro.virtual_ab.enabled` after boot (§4.3), never assume here.

4.2 [AGENT] Launch the cvd (daemon)

```
# [AGENT] inside cuttlefish – daemonised launch (privileged
    container already up).
podman exec cuttlefish bash -lc '
    cd "${HOME:-/cuttlefish}"
    HOME="$PWD" ./bin/launch_cvd --daemon
    ./bin/adb wait-for-device
    ./bin/adb shell getprop sys.boot_completed
'
```

A non-zero exit / no `cvd-ebr` bridge here means the privileged flags from §2.3 did not take (most often a missing device or group) — that is a real FAIL surfaced in `podman logs cuttlefish`, never fake-passed (§11.4.6).

4.3 [AGENT] Drive the A/B + auto-rollback validation

The agent points the existing validation harness at the running `cvd`. Two paths:

Path A (preferred) — run the project's `tier2_cuttlefish_ab.sh` harness (it detects A/B variant, applies the OTA, asserts the slot flip, then does the headline corrupt-slot **auto-rollback** — all with captured evidence + honest FAIL on any unconfirmed step):

```
# [AGENT] inside cuttlefish (or against the cvd from nezha host
    adb), point the
# harness at the running cvd. The harness lives in the checkout;
    run it where it
# can reach ./bin/adb of the cvd. Evidence lands under docs/qa/
    <run-id>/cuttlefish_ab/.
podman exec cuttlefish bash -lc '
    cd "${HOME:-/cuttlefish}"
    # The cvd host package already provides bin/adb + bin/
        update_device.py;
    # the harness uses ro.boot.slot_suffix + update_engine_client +
        bootctl.
    HELIX_CF_DIR="$PWD/.." HELIX_CF_EVIDENCE="$PWD/evidence" \
        /path/to/helix_ota/tests/emulator/tier2_cuttlefish_ab.sh
'
```

The harness (`tests/emulator/tier2_cuttlefish_ab.sh`) already encodes the UNCONFIRMED: discipline (§11.4.6/§11.4.123): it **DETECTS** Virtual-A/B vs legacy A/B (`ro.virtual_ab.enabled`), **ATTEMPTS** the documented OTA-apply (`update_device.py` on an `ota.zip`) and the documented corrupt-slot mechanism (`bootctl set-slot-as-unbootable` → fallback bounded `dd` to the inactive `system<slot>` partition), and **FAILS HONESTLY** if any step does not reproduce on this host — never a fake PASS.

Path B (manual, if the harness needs host-specific adaptation) — the agent drives the same sequence by hand inside cuttlefish, capturing each artifact:

1. baseline `ro.boot.slot_suffix` (→ `getprop_before.txt`)
2. build/fetch a signed OTA for `aosp_cf_x86_64_only_phone-userdebug`, apply via `update_device.py` (→ `apply.log`), confirm `update_engine_client --follow` reaches `UPDATED_NEED_REBOOT` (→ `update_engine.txt`)
3. `adb reboot`, assert `ro.boot.slot_suffix` **FLIPPED** (→ `slot_after.txt`) + `dm-verity present` (`dmesg | grep -i verity` → `verity.txt`)
4. corrupt the now-inactive slot (variant-correct: `bootctl set-slot-as-unbootable` or `bounded dd`), set it active, `adb reboot`, assert **AUTO-ROLLBACK** to the known-good slot (→ `slot_after_rollback.txt` + `rollback_trace.txt`)

UNCONFIRMED: (carried from `CUTTLEFISH_TIER2.md`, established at run time) — the exact OTA-apply invocation (`update_device.py` vs a `cvd` subcommand), the Virtual-A/B COW/snapuserd corrupt path vs the legacy direct-partition write, and `bootctl` shell availability. The harness attempts each and FAILs honestly; do not hardcode a guess.

4.4 [AGENT] Evidence capture

Every PASS cites a captured artifact (§11.4.69 `ab_pass_with_evidence` shape). The agent collects the harness's evidence dir out of the container to a durable, committed path:

```
# [AGENT] pull evidence out of the container to a durable docs/qa
path.
RUN_ID="$(date -u +%Y%m%dT%H%M%S)-cuttlefish-ab"
mkdir -p ~/helix_ota/docs/qa/${RUN_ID}
podman cp cuttlefish:/cuttlefish/evidence/. ~/helix_ota/docs/qa/${RUN_ID}/
# curated REPORT.md cites: slot_before, slot_after (FLIPPED),
verity, slot_after_rollback
# (AUTO-ROLLBACK to known-good), rollback_trace – §11.4.107/
§11.4.108/§11.4.69.
```

The durable evidence files carry the §11.4.155 project-name prefix where they are recordings; the curated `REPORT.md` under `docs/qa/<run-id>/` is the committed proof (§11.4.83). Raw `cvd` logs stay gitignored (§11.4.128).

5. Honest boundary (§11.4.6) — what a green here does and does NOT mean

- A PASS from §4 means: on nezha, the real Android `update_engine` applied an OTA to the inactive slot, the active slot **FLIPPED** on reboot, `dm-verity` was present, and a deliberately-corrupted slot **AUTO-ROLLED-BACK** to the known-good slot — all with captured evidence. THAT is the real-A/B fidelity the QEMU A/B-virt tier and the NON-A/B RK3588 boards cannot reach.

- A PASS here does **NOT** prove the RK3588 / Orange Pi 5 Max vendor HAL / U-Boot slot-switch / real-partition dm-verity (Tier-3 / OTA-004 — different boot stack); Cuttlefish is the closest hardware-free proxy, not the physical board.
- **Until §§4–6 actually run with captured slot-flip + rollback evidence, F112 / OTA-003 stays integration-pending — NOT a real-A/B PASS.** This runbook being ready is not the run being done (§11.4.6 — "ready to run" ≠ "ran").
- Every UNCONFIRMED: above is a **verify-at-run-time** item, established as FACT on first bring-up (the device mounts, Virtual-A/B-vs-legacy, the corrupt-slot mechanism), never asserted from this document.

6. Teardown

```
# [AGENT] stop the cvd cleanly inside the container (§11.4.14
    quiescence).
podman exec cuttlefish bash -lc 'cd "${HOME:-/cuttlefish}" &&
    HOME="$PWD" ./bin/stop_cvd || true'

# [OPERATOR] remove the privileged container (it was started under
    sudo on nezha).
sudo podman rm -f cuttlefish
```

Reversibility (FACT, from CUTTLEFISH_ROOTFUL_EXCEPTION.md §6): no persistent host mutation beyond reversible config — the privileged window is the container's lifetime only; the only host-state changes are the (transient) loaded kernel modules and the (reversible gpasswd -d) group membership. The A/B slot-switch + corrupt-slot rollback happen **inside** the virtual device, never on nezha itself (§11.4.133 — the "target" is the cvd guest, not the host).

Operator-vs-agent step split (at a glance)

Step	Who	Privileged?
§1.3 Build the Cuttlefish container image (podman build)	AGENT	no (rootless)
§2.1 Verify/load vhost_vsock/ vhost_net; create /dev/vsock if absent	OPERATOR	yes (sudo)
§2.2 One-time group membership (usermod -aG kvm, cvdnetwork, render)	OPERATOR	yes (sudo)
§2.3 The --privileged --network host container run	OPERATOR	yes (sudo)
§3 Handoff (confirm cuttlefish Up)	OPERATOR → AGENT	—
§4.1 Extract staged assets inside the container	AGENT	no (inside already-privileged container)

Step	Who	Privileged?
§4.2 <code>launch_cvd --daemon</code>	AGENT	no (inside already-privileged container)
§4.3 A/B apply + slot-flip + auto-rollback validation	AGENT	no
§4.4 Evidence capture to <code>docs/qa/<run-id>/</code>	AGENT	no
§6 <code>stop_cvd</code>	AGENT	no
§6 <code>podman rm -f cuttlefish</code>	OPERATOR	yes (sudo)

The privileged surface is **exactly three operator commands** (§2.1 conditional, §2.2 one-time, §2.3 the run) + the one teardown remove — everything else is agent-driven.